Gerber, Green & Larimer (2008), *Social Pressure and Voter Turnout: Evidence from a Large-Scale Field Experiment* — SH Day 1: Model Evaluation

Machine-Learning Workshop teaching example

2026-07-20

1 The study

Citation. Alan S. Gerber, Donald P. Green, and Christopher W. Larimer (2008). "Social Pressure and Voter Turnout: Evidence from a Large-Scale Field Experiment." *American Political Science Review* 102(1): 33–48.

The design. This is one of the most famous field experiments in modern social science. Ahead of the **August 2006 primary election in Michigan**, the authors randomly assigned **344,084 registered voters in 180,002 households** — with randomization at the *household* level — to a no-mail **Control** condition or to one of four mailings sent shortly before the election, each turning up the dial of social pressure:

1. **Civic Duty.** A mailing that simply appeals to the recipient's sense of civic duty ("Remember your rights and responsibilities as a citizen. Remember to vote."). This is the baseline mobilization message.
2. **Hawthorne.** Adds mild surveillance: recipients are told they are part of a study and that "You are being studied!" — their turnout will be examined by researchers via public records. No information is disclosed to anyone else.
3. **Self.** Adds *self*-monitoring: the mailing lists the recipient's own household's past turnout (who voted in 2004, who did not) and promises an updated mailing after the election showing whether each member voted.
4. **Neighbors.** The maximal-pressure treatment: the mailing lists the recent turnout of the recipient's household *and of their neighbors*, and promises a follow-up mailing telling the whole neighborhood who voted.

Because voting in the United States is a matter of public record, the threat to publicize turnout is credible, and the design lets the authors measure — with experimental precision — how much *social pressure*, over and above a civic-duty appeal, moves people to the polls.

Why it is a landmark. The result was enormous by the standards of the get-out-the-vote literature: the Neighbors mailing raised turnout by **8.1 percentage points** off a control base of 29.7% — an effect comparable to face-to-face canvassing, achieved with a single piece of mail costing pennies. The paper has shaped both the science of voter mobilization (spawning a large literature on social-pressure interventions) and its practice (variants of the Neighbors letter have been used, sometimes controversially, in real campaigns). It is also a model of design-based inference: with random assignment, a simple difference in means is an unbiased causal estimate.

The published quantities we reproduce. Two headline results:

- **Table 2**: the percentage voting in the August 2006 primary within each of the five experimental groups (Control 29.7%; Civic Duty 31.5%; Hawthorne 32.2%; Self 34.5%; Neighbors 37.8%).
- **Table 3, model (a)**: OLS of the turnout indicator on the four treatment dummies (Control as reference), with robust standard errors **clustered on household** — Civic Duty .018, Hawthorne .026, Self .049, Neighbors .081, all with clustered SEs of .003 and $p < .001$; $N = 344{,}084$ individuals.

```r
ggl <- read.csv(paste0(
  "https://raw.githubusercontent.com/bdesmarais/",
  "intro-ml-statistical-horizons-4day/main/",
  "tutorials/day1_intro_and_evaluation/",
  "gerber_green_larimer_turnout/data/",
  "ggl_individual.csv"))
dim(ggl)
```

```
## [1] 344084     18
```

2 Data and codebook

The unit of analysis is the **individual registered voter**, nested in households. The file has 344,084 rows and 18 columns.

```r
codebook <- data.frame(
  Variable = c("sex", "yob", "g2000, g2002, g2004",
               "p2000, p2002, p2004", "treatment", "cluster",
               "voted", "hh_id", "hh_size", "numberofnames",
               "p2004_mean", "g2004_mean", "X", "origorder"),
  Description = c(
    "male / female",
    "year of birth",
    "voted in the 2000/2002/2004 GENERAL election (yes/no)",
    "voted in the 2000/2002/2004 PRIMARY election (yes/no)",
    "experimental condition (Control + 4 mailings)",
    "grouping identifier from the assignment procedure",
    "OUTCOME: voted in the August 2006 primary (0/1)",
    "household identifier (randomization unit)",
    "number of registered voters in the household",
    "number of names shown on the Neighbors mailing",
    "household mean of p2004 (pre-treatment)",
    "household mean of g2004 (pre-treatment)",
    "empty unnamed column in the source file (dropped)",
    "original row order in the replication file"))
knitr::kable(codebook, caption = "Codebook for ggl_individual.csv.")
```

Table 1: Codebook for ggl_individual.csv.

Variable	Description
sex	male / female
yob	year of birth
g2000, g2002, g2004	voted in the 2000/2002/2004 GENERAL election (yes/no)
p2000, p2002, p2004	voted in the 2000/2002/2004 PRIMARY election (yes/no)
treatment	experimental condition (Control + 4 mailings)
cluster	grouping identifier from the assignment procedure
voted	OUTCOME: voted in the August 2006 primary (0/1)
hh_id	household identifier (randomization unit)
hh_size	number of registered voters in the household
numberofnames	number of names shown on the Neighbors mailing
p2004_mean	household mean of p2004 (pre-treatment)
g2004_mean	household mean of g2004 (pre-treatment)
X	empty unnamed column in the source file (dropped)

Variable	Description
origorder	original row order in the replication file

Cleaning. Three quirks of the source file need attention before any analysis. First, the `treatment` strings carry a **leading space** (e.g. `" Control"`). Second, the vote-history columns mix capitalization and can carry stray whitespace (`p2004` is recorded as `"Yes"`/`"No"` while the others are lowercase `"yes"`/`"no"`); we inspect the raw values, then `trimws()` and lowercase everything and convert to 0/1 indicators. Third, there is an empty unnamed column (read in as `X`), which we drop.

```
# Inspect the raw coding quirks before touching anything.
table(ggl$treatment)          # note the leading space in every level
```

```
##
##  Civic Duty     Control   Hawthorne   Neighbors        Self
##       38218      191243       38204       38201       38218
```

```
table(ggl$p2004)              # 'Yes'/'No' capitalization differs from the rest
```

```
##
##     No    Yes
## 205934 138150
```

```
ggl$X <- NULL                                       # drop the empty column
ggl$treatment <- trimws(ggl$treatment)              # strip the leading space
hist_vars <- c("g2000", "g2002", "g2004", "p2000", "p2002", "p2004")
for (v in hist_vars)                                # 'yes'/'Yes ' etc. -> 0/1
  ggl[[v]] <- as.integer(trimws(tolower(ggl[[v]])) == "yes")
ggl$treatment <- relevel(factor(ggl$treatment), ref = "Control")
ggl$age    <- 2006 - ggl$yob                        # age at the 2006 primary
ggl$female <- as.integer(ggl$sex == "female")
stopifnot(!anyNA(ggl[c("voted", "treatment", "age", hist_vars)]))
```

One vote-history variable deserves a flag: **`g2004` equals 1 for every single row** (344,084 of 344,084). That is by design — the authors restricted the experimental universe to registered voters who had voted in the 2004 general election. A constant carries no information, so `g2004` is documented here but excluded from every model below; the five informative history indicators are `g2000`, `g2002`, `p2000`, `p2002`, `p2004`.

3 Descriptive analysis

Before reproducing anything we get to know the data thoroughly. With an experiment the stakes of EDA are slightly different than with observational data: beyond documenting distributions, the key descriptive product is the **balance check** — randomization should make the five groups look alike on everything determined *before* treatment, and we verify that it does.

3.1 Experimental group sizes

```
grp_n <- as.data.frame(table(Condition = ggl$treatment),
                       responseName = "Individuals")
grp_hh <- tapply(ggl$hh_id, ggl$treatment,
                 function(x) length(unique(x)))
grp_n$Households <- as.integer(grp_hh[as.character(grp_n$Condition)])
grp_n$Share <- round(100 * grp_n$Individuals / nrow(ggl), 1)
knitr::kable(grp_n, caption =
```

```
  "Experimental group sizes: individuals, households, and percent of sample.")
```

Table 2: Experimental group sizes: individuals, households, and percent of sample.

Condition	Individuals	Households	Share
Control	191243	99999	55.6
Civic Duty	38218	20001	11.1
Hawthorne	38204	20002	11.1
Neighbors	38201	20000	11.1
Self	38218	20000	11.1

The design is unbalanced on purpose: about 56% of the sample is Control, and each mailing goes to roughly 38,000 individuals in about 20,000 households. In total there are 180,002 households — matching the paper's 180,002.

3.2 Turnout by experimental condition

The single most important descriptive picture: the share voting in the August 2006 primary in each condition, with 95% confidence intervals. (With group sizes in the tens of thousands the CIs are so tight they are barely visible — one of the luxuries of a 344,084-person experiment.)

```
tb <- aggregate(voted ~ treatment, ggl, mean)
tb$n  <- as.integer(table(ggl$treatment)[as.character(tb$treatment)])
tb$se <- sqrt(tb$voted * (1 - tb$voted) / tb$n)
tb$lo <- tb$voted - 1.96 * tb$se
tb$hi <- tb$voted + 1.96 * tb$se
tb$treatment <- factor(tb$treatment, levels = c(
  "Control", "Civic Duty", "Hawthorne", "Self", "Neighbors"))
ggplot(tb, aes(x = treatment, y = voted)) +
  geom_col(fill = "steelblue", width = 0.65) +
  geom_errorbar(aes(ymin = lo, ymax = hi), width = 0.15) +
  geom_text(aes(label = sprintf("%.1f%%", 100 * voted)),
            vjust = -1.1, size = 3.2) +
  scale_y_continuous(labels = function(x) sprintf("%.0f%%", 100 * x),
                     limits = c(0, 0.45)) +
  labs(x = NULL, y = "Voted in Aug 2006 primary",
       title = "Turnout rises with the social-pressure dosage") +
  theme_minimal(base_size = 11)
```

The bar chart *is* the experiment's result: turnout climbs monotonically from Control (29.7%) through Civic Duty, Hawthorne, and Self to Neighbors (37.8%), exactly the ordering of social-pressure intensity.

3.3 Vote history

The five informative history indicators record turnout in the 2000 and 2002 general elections and the 2000, 2002, and 2004 primaries.

```
hist5 <- c("g2000", "g2002", "p2000", "p2002", "p2004")
vh <- data.frame(
  Election = c("General 2000", "General 2002", "Primary 2000",
               "Primary 2002", "Primary 2004"),
  Voted_pct = round(100 * sapply(ggl[hist5], mean), 1))
```

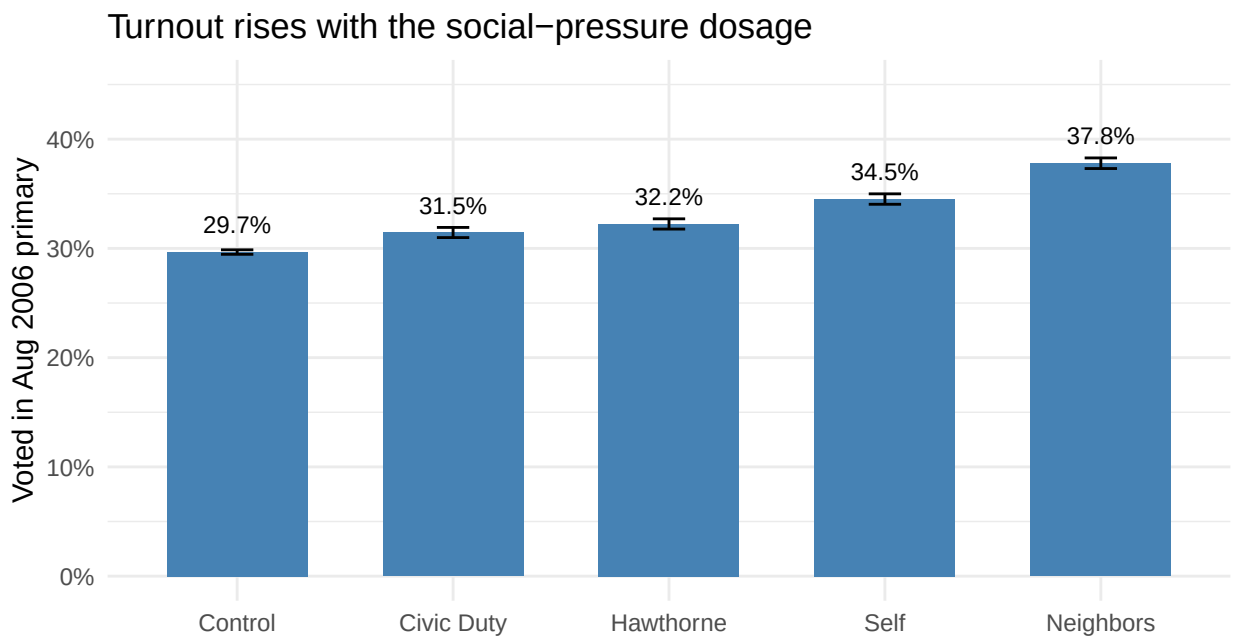


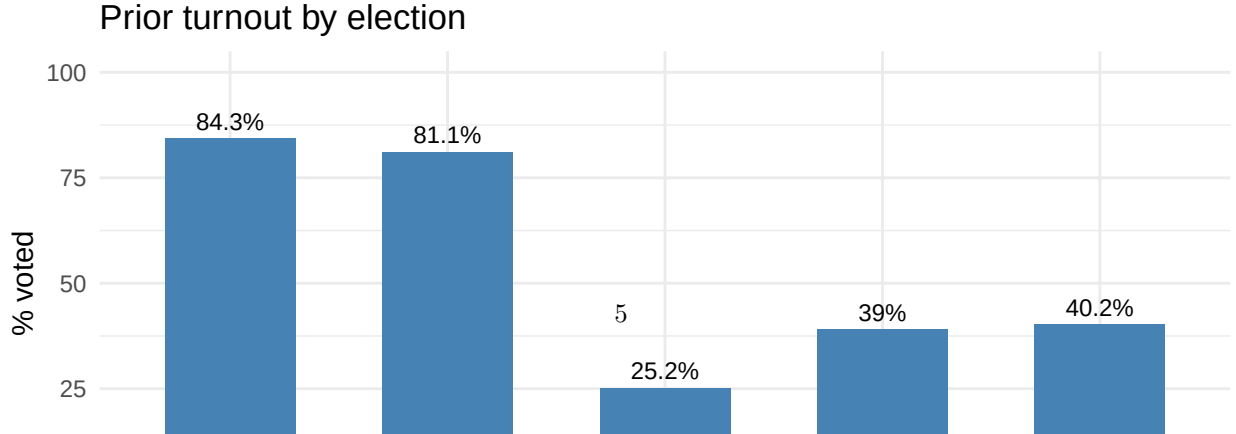
Figure 1: Turnout in the August 2006 primary by experimental condition, with 95 percent confidence intervals (barely visible at these sample sizes). Turnout climbs monotonically with the social-pressure dosage.

```
knitr::kable(vh, row.names = FALSE, caption =
  "Share voting in each prior election (g2004 = 100% by sample construction).")
```

Table 3: Share voting in each prior election (g2004 = 100% by sample construction).

Election	Voted_pct
General 2000	84.3
General 2002	81.1
Primary 2000	25.2
Primary 2002	39.0
Primary 2004	40.2

```
vh$Election <- factor(vh$Election, levels = vh$Election)
ggplot(vh, aes(x = Election, y = Voted_pct)) +
  geom_col(fill = "steelblue", width = 0.6) +
  geom_text(aes(label = paste0(Voted_pct, "%")), vjust = -0.5, size = 3.2) +
  ylim(0, 100) +
  labs(x = NULL, y = "% voted", title = "Prior turnout by election") +
  theme_minimal(base_size = 11)
```



```
  geom_col(fill = "steelblue", width = 1) +
  labs(x = "Age in 2006", y = "Registered voters",
       title = "Age distribution") +
  theme_minimal(base_size = 11)
```

Figure 3: Age distribution at the time of the 2006 primary. The registration-list sample skews middle-aged and older.

```
summary(ggl$age)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   20.00   41.00   50.00   49.79   59.00  106.00
```

```
age_to <- aggregate(voted ~ age, ggl, mean)
age_to$n <- as.integer(table(ggl$age)[as.character(age_to$age)])
age_to <- age_to[age_to$n >= 200, ]
ggplot(age_to, aes(x = age, y = voted)) +
  geom_point(colour = "steelblue", size = 1) +
  geom_smooth(method = "loess", se = FALSE, colour = "grey30",
              linewidth = 0.7) +
  scale_y_continuous(labels = function(x) sprintf("%.0f%%", 100 * x)) +
  labs(x = "Age in 2006", y = "Turnout, Aug 2006 primary",
       title = "Turnout by age is strongly nonlinear") +
  theme_minimal(base_size = 11)
```

Turnout rises steeply with age from the 20s into the 60s–70s and then bends back down — a strong, visibly **nonlinear** relationship. Keep this plot in mind: in Section 5 the degree of a polynomial in age becomes our bias–variance knob.

3.5 Turnout by past-vote count

Summing the five informative history indicators gives each voter a **past-vote count** from 0 to 5 — a one-number habit measure.

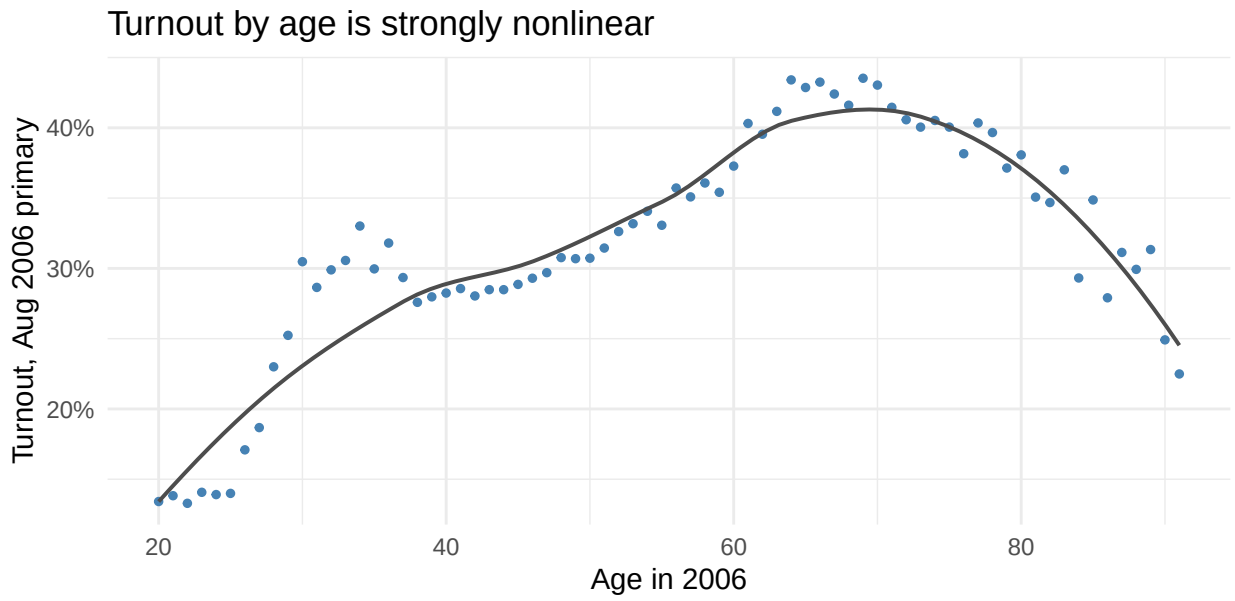


Figure 4: Turnout in the 2006 primary by year of age (ages with at least 200 voters). The relationship is strong and curved – rising steeply from the 20s to the 60s, then flattening and turning down at advanced ages. A straight line in age would miss this shape; polynomials will pick it up in Section 5.

```
ggl$past_votes <- rowSums(ggl[hist5])
pc <- aggregate(voted ~ past_votes, ggl, mean)
pc$n <- as.integer(table(ggl$past_votes))
knitr::kable(data.frame(Past_votes = pc$past_votes,
                        N = format(pc$n, big.mark = ","),
                        Turnout_2006_pct = round(100 * pc$voted, 1)),
             caption = "2006 primary turnout by past-vote count (0-5 prior elections).")
```

Table 4: 2006 primary turnout by past-vote count (0-5 prior elections).

Past_votes	N	Turnout_2006_pct
0	21,784	8.7
1	26,060	16.8
2	80,436	23.7
3	136,031	33.3
4	65,852	44.1
5	13,921	65.3

```
ggplot(pc, aes(x = past_votes, y = voted)) +
  geom_col(fill = "steelblue", width = 0.6) +
  geom_text(aes(label = sprintf("%.0f%%", 100 * voted)),
            vjust = -0.5, size = 3.2) +
  scale_y_continuous(labels = function(x) sprintf("%.0f%%", 100 * x),
                     limits = c(0, 0.75)) +
  scale_x_continuous(breaks = 0:5) +
  labs(x = "Prior elections voted in (of 5)", y = "Turnout, Aug 2006",
       title = "Voting is a habit") +
```

```
  theme_minimal(base_size = 11)
```

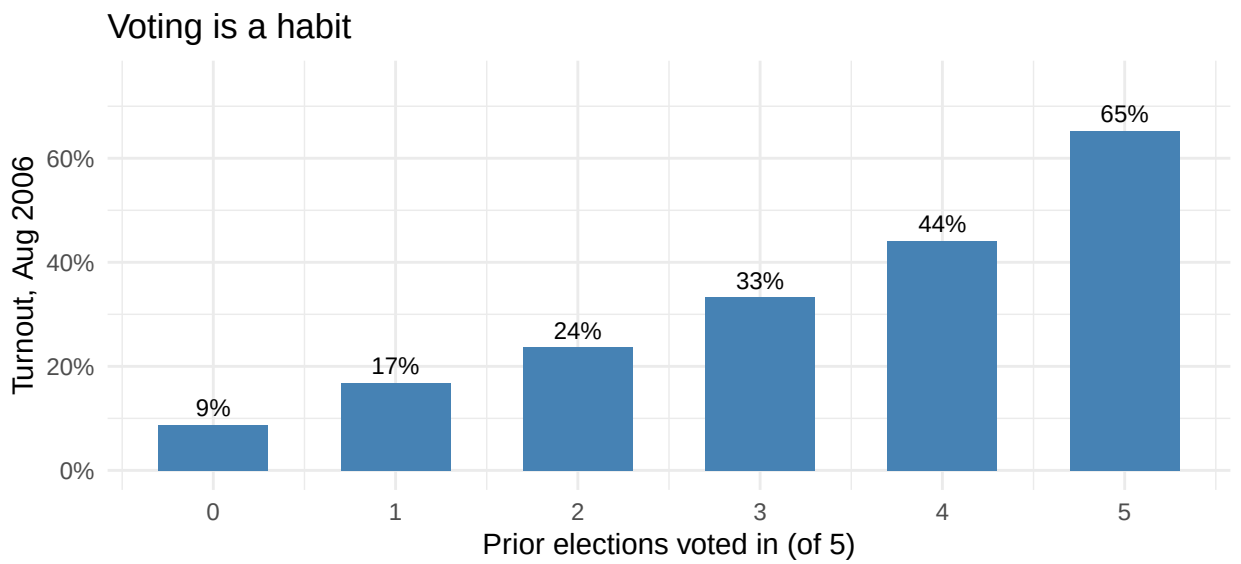


Figure 5: Turnout by past-vote count. Voting is habitual: those who voted in all five prior elections turn out at several times the rate of those who voted in none.

The gradient is enormous: turnout runs from 9% among those with no prior votes to 65% among those who voted in all five prior elections. **Past behavior dwarfs any treatment effect as a predictor** — the Neighbors mailing moves turnout 8 points; vote history spans more than 40. This asymmetry is the engine of the predictive modeling in Section 5.

3.6 Household size

```
hh_tab <- as.data.frame(table(hh_size = ggl$hh_size),
                        responseName = "Individuals")
hh_tab$Pct <- round(100 * hh_tab$Individuals / nrow(ggl), 2)
knitr::kable(hh_tab, caption =
  "Individuals by household size (number of registered voters at the address).")
```

Table 5: Individuals by household size (number of registered voters at the address).

hh_size	Individuals	Pct
1	47834	13.90
2	214086	62.22
3	57474	16.70
4	20916	6.08
5	3315	0.96
6	402	0.12
7	49	0.01
8	8	0.00

Almost everyone lives in a one- or two-voter household — a consequence of the authors' screening of the registration list. Household structure matters for two reasons: treatment was assigned at the household level

(everyone in a household gets the same condition), and household members' turnout is correlated. Both facts will force us to **cluster** — standard errors on households in the reproduction, and train/test splits on households in the extension.

3.7 Balance check: covariates across conditions

The whole point of randomization is that pre-treatment covariates should be (near-)identical across conditions. We check age, sex, household size, and all five history indicators.

```
bal_vars <- c("age", "female", "hh_size", hist5)
bal <- sapply(split(ggl[bal_vars], ggl$treatment), colMeans)
bal <- bal[, c("Control", "Civic Duty", "Hawthorne", "Self", "Neighbors")]
knitr::kable(round(bal, 3), caption =
  "Balance check: pre-treatment covariate means by experimental condition.")
```

Table 6: Balance check: pre-treatment covariate means by experimental condition.

	Control	Civic Duty	Hawthorne	Self	Neighbors
age	49.814	49.659	49.705	49.793	49.853
female	0.499	0.500	0.499	0.500	0.500
hh_size	2.184	2.189	2.180	2.181	2.188
g2000	0.843	0.842	0.844	0.840	0.842
g2002	0.811	0.811	0.813	0.811	0.811
p2000	0.252	0.254	0.250	0.251	0.251
p2002	0.389	0.389	0.394	0.392	0.387
p2004	0.400	0.399	0.403	0.402	0.407

Balance holds to the third decimal place essentially everywhere: mean age varies by a few tenths of a year across groups, and the history indicators by a few tenths of a percentage point. This is what a well-executed randomization looks like, and it is why the raw turnout differences in the bar chart above can be read directly as causal effects.

3.8 Co-occurrence of the vote-history indicators

Finally, how do the five history indicators relate to each other and to the outcome? For binary variables the Pearson correlation is the ϕ coefficient.

```
cm <- cor(ggl[c(hist5, "voted")])
cl <- as.data.frame(as.table(cm))
names(cl) <- c("V1", "V2", "r")
ggplot(cl, aes(V1, V2, fill = r)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = sprintf("%.2f", r)), size = 2.8) +
  scale_fill_gradient2(low = "#b2182b", mid = "white", high = "#2166ac",
                       midpoint = 0, limits = c(-1, 1)) +
  labs(x = NULL, y = NULL,
       title = "Vote-history co-occurrence (phi coefficients)") +
  theme_minimal(base_size = 10)
```

All correlations are positive — habitual voters vote everywhere — but they are moderate (-0.07 to 0.40 among the history indicators), with primaries correlating more strongly with other primaries than with generals. No pair is close to collinear, so all five can enter a model together. The primary indicators (especially `p2004`, $\phi = 0.16$ with the outcome) are the strongest single predictors of 2006 primary turnout — like predicts like.

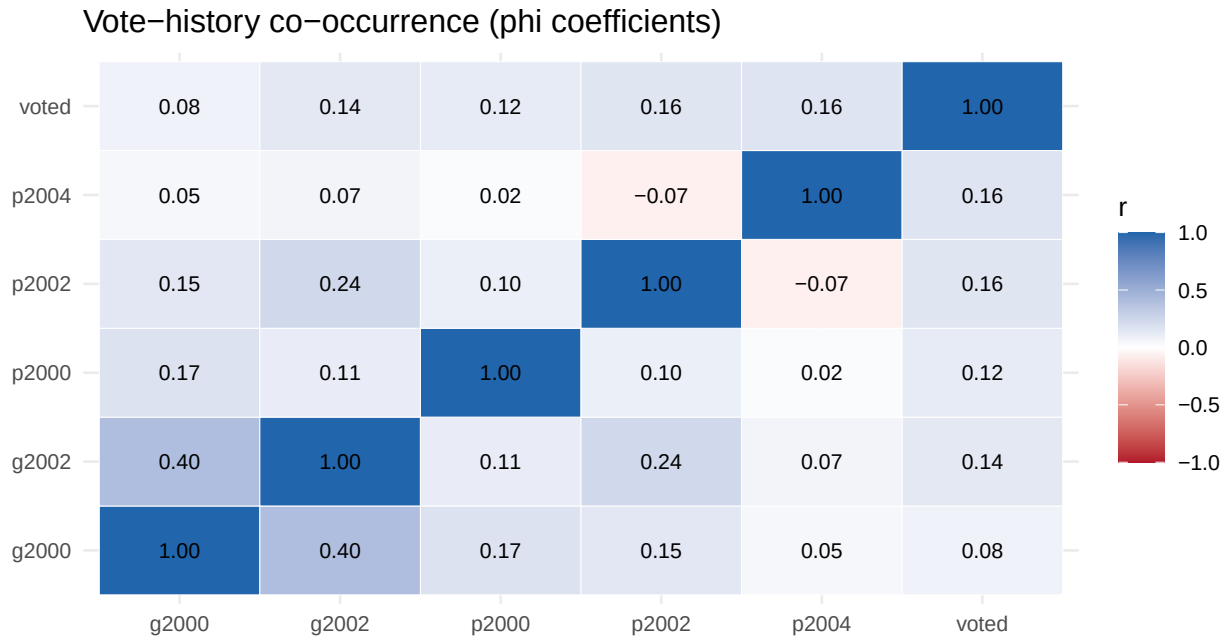


Figure 6: Correlation (phi) matrix of the five vote-history indicators and 2006 turnout. Same-type elections cohere most strongly; every history indicator is positively related to the 2006 outcome.

3.9 Reading the descriptives as a whole

Pulling the EDA together: the outcome is a **moderately rare binary event** (base rate ~32%), the five conditions are cleanly balanced on everything pre-treatment (so raw group differences are causal), and turnout climbs monotonically with social-pressure dosage. On the predictive side, the strongest signals in the data are **not** the experimental treatments: past turnout (a 40+-point gradient) and age (a strong, curved profile) dominate the 2–8-point treatment contrasts. And the household structure — shared treatment and correlated behavior within addresses — warns us that both inference and evaluation must respect the household clustering. All three observations script what follows.

4 Reproduction of the published results

4.1 Table 2: percentage voting by experimental group

The paper's Table 2 reports the share voting in the August 2006 primary in each group. We compute it and set it beside the published values.

```
pub_grp <- c("Control", "Civic Duty", "Hawthorne", "Self", "Neighbors")
pub_pct <- c(29.7, 31.5, 32.2, 34.5, 37.8)
pub_N   <- c(191243, 38218, 38204, 38218, 38201)

comp_pct <- round(100 * tapply(ggl$voted, ggl$treatment, mean)[pub_grp], 1)
comp_N   <- as.integer(table(ggl$treatment)[pub_grp])

t2 <- data.frame(Group = pub_grp,
                 Published_pct = pub_pct, Computed_pct = as.numeric(comp_pct),
                 Published_N = pub_N, Computed_N = comp_N)
knitr::kable(t2, row.names = FALSE, caption =
  "Table 2 reproduction: percentage voting and N by experimental group, published vs.
  ↪  computed.")
```

Table 7: Table 2 reproduction: percentage voting and N by experimental group, published vs. computed.

Group	Published_pct	Computed_pct	Published_N	Computed_N
Control	29.7	29.7	191243	191243
Civic Duty	31.5	31.5	38218	38218
Hawthorne	32.2	32.2	38204	38204
Self	34.5	34.5	38218	38218
Neighbors	37.8	37.8	38201	38201

```
# --- Reproduction gate 1: Table 2 must match exactly at printed precision ---
stopifnot(max(abs(as.numeric(comp_pct) - pub_pct)) < 1e-9,
          identical(comp_N, as.integer(pub_N)))
```

Every percentage matches the published value exactly at the paper's printed precision, and every group N matches exactly. The Neighbors–Control contrast is 8.1 percentage points — the paper's headline 8.1-point effect, straight from the raw means.

4.2 Table 3, model (a): OLS with household-clustered standard errors

Table 3(a) regresses the turnout indicator on the four treatment dummies (Control is the reference). Because treatment was randomized by household and household members' outcomes are correlated, the paper reports robust standard errors **clustered on household**; we do the same via `sandwich::vcovCL()`.

```
m_t3 <- lm(voted ~ treatment, data = ggl)
ct   <- coeftest(m_t3, vcov = vcovCL(m_t3, cluster = ~ hh_id))
ct
```

```
##
## t test of coefficients:
##
##                       Estimate Std. Error  t value  Pr(>|t|)
## (Intercept)          0.2966383  0.0013096 226.5172 < 2.2e-16 ***
## treatmentCivic Duty  0.0178993  0.0032366   5.5302 3.200e-08 ***
## treatmentHawthorne   0.0257363  0.0032579   7.8997 2.804e-15 ***
## treatmentNeighbors   0.0813099  0.0033696  24.1308 < 2.2e-16 ***
## treatmentSelf        0.0485132  0.0033000  14.7009 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
rows    <- paste0("treatment", c("Civic Duty", "Hawthorne", "Self", "Neighbors"))
pub_b   <- c(0.018, 0.026, 0.049, 0.081)
pub_se  <- c(0.003, 0.003, 0.003, 0.003)
comp_b  <- ct[rows, "Estimate"]
comp_se <- ct[rows, "Std. Error"]

t3 <- data.frame(
  Term = c("Civic Duty", "Hawthorne", "Self", "Neighbors"),
  Published_b  = pub_b,  Computed_b  = round(comp_b, 4),
  Published_SE = pub_se, Computed_SE = round(comp_se, 4),
  p = signif(ct[rows, "Pr(>|t|)"], 2))
knitr::kable(t3, row.names = FALSE, caption =
  "Table 3, model (a) reproduction: OLS treatment effects with household-clustered SEs,
  ↪  published vs. computed. N = 344,084 individuals; all p < .001.")
```

Table 8: Table 3, model (a) reproduction: OLS treatment effects with household-clustered SEs, published vs. computed. N = 344,084 individuals; all p < .001.

Term	Published_b	Computed_b	Published_SE	Computed_SE	p
Civic Duty	0.018	0.0179	0.003	0.0032	0
Hawthorne	0.026	0.0257	0.003	0.0033	0
Self	0.049	0.0485	0.003	0.0033	0
Neighbors	0.081	0.0813	0.003	0.0034	0

```
# --- Reproduction gate 2: coefficients and clustered SEs must round to the
# --- published three-decimal values.
stopifnot(max(abs(round(comp_b, 3)  - pub_b))  < 1e-9,
          max(abs(round(comp_se, 3) - pub_se)) < 1e-9,
          nobs(m_t3) == 344084)
```

The fresh fit gives 0.0179, 0.0257, 0.0485, and 0.0813 with clustered SEs between 0.0032 and 0.0034 — a **digit-perfect match at the paper's printed precision** (.018/.026/.049/.081, all SEs .003), on the full 344,084 individuals, all $p < .001$. Both `stopifnot()` gates pass: **the reproduction gate is passed**, and this fitted experiment is the anchor for everything below.

5 Day 1: Model evaluation — from average effects to individual predictions

The experiment answers a *causal* question superbly: sending the Neighbors mailing raises turnout by 8.1 points **on average**. Day 1 asks the *predictive* question: given what we know about a registered voter — their vote history, age, sex, household, and which mailing (if any) they received — **can we predict whether that particular person will vote?** The two questions are different, and the EDA told us why: the treatments shift turnout by 2–8 points, but vote history spans 40+ points. A good *predictor* of turnout may lean mostly on variables the *experiment* never manipulated. We build the standard Day-1 evaluation toolkit — train/test splits, cross-validation, the bias–variance tradeoff, ROC and precision–recall, calibration — on top of the reproduced experiment.

5.1 The model ladder and the metrics

We compare four logistic-regression models of `voted`, each adding information:

1. **Baseline** — intercept only: predict the base rate for everyone.
2. **Treatment only** — the experiment's information set: the four dummies.
3. **+ Vote history** — adds the five informative history indicators.
4. **Full** — adds age (with a quadratic), sex, and household size.

Because the outcome is binary we score models with **log-loss** (average negative log-likelihood of the observed outcome under the predicted probability — lower is better, and it punishes confident wrong predictions hard), **AUC** (the probability a random voter is ranked above a random non-voter — 0.5 is coin-flipping), and later the **Brier score** for calibration.

```
logloss <- function(y, p) {
  p <- pmin(pmax(p, 1e-12), 1 - 1e-12)
  -mean(y * log(p) + (1 - y) * log(1 - p))
}
auc_of <- function(y, p) {
  if (length(unique(p)) < 2) return(0.5)   # constant predictor: chance
```

```
  as.numeric(pROC::auc(y, p, quiet = TRUE, direction = "<"))
}
f_treat <- voted ~ treatment
f_hist  <- voted ~ treatment + g2000 + g2002 + p2000 + p2002 + p2004
f_full  <- update(f_hist, . ~ . + poly(age, 2) + female + hh_size)
```

5.2 An honest train/test split — by household

We hold out 30% of the data as a test set. But we do **not** sample individuals: we sample **households**. Treatment is assigned by household and turnout is correlated within households (spouses tend to vote — or not — together). If we split individuals, one member of a household could land in training and the other in test; the model would be graded partly on households it has effectively already seen, and the test score would be **optimistically biased** by this leakage. Splitting on the randomization/clustering unit keeps the test set genuinely unseen. This is the general rule: *split on the unit whose observations are dependent.*

```
set.seed(2006)
hh_ids   <- unique(ggl$hh_id)
tr_hh    <- sample(hh_ids, round(0.70 * length(hh_ids)))
in_train <- ggl$hh_id %in% tr_hh
train <- ggl[in_train, ]
test  <- ggl[!in_train, ]
c(train_ind = nrow(train), test_ind = nrow(test),
  train_hh = length(unique(train$hh_id)),
  test_hh  = length(unique(test$hh_id)))
```

```
## train_ind  test_ind  train_hh   test_hh
##    240798    103286    126001     54001
```

```
p_base  <- rep(mean(train$voted), nrow(test))       # model 1: base rate
m_tr    <- glm(f_treat, train, family = binomial)   # model 2
m_hi    <- glm(f_hist,  train, family = binomial)   # model 3
m_fu    <- glm(f_full,  train, family = binomial)   # model 4
p_tr <- predict(m_tr, test, type = "response")
p_hi <- predict(m_hi, test, type = "response")
p_fu <- predict(m_fu, test, type = "response")

split_tab <- data.frame(
  Model = c("1. Baseline (intercept)", "2. Treatment only",
            "3. + Vote history", "4. Full (+ age, sex, hh size)"),
  LogLoss = round(c(logloss(test$voted, p_base), logloss(test$voted, p_tr),
                    logloss(test$voted, p_hi),   logloss(test$voted, p_fu)), 4),
  AUC     = round(c(auc_of(test$voted, p_base), auc_of(test$voted, p_tr),
                    auc_of(test$voted, p_hi),   auc_of(test$voted, p_fu)), 3))
knitr::kable(split_tab, caption =
  "Held-out (30% of households) test performance of the model ladder.")
```

Table 9: Held-out (30% of households) test performance of the model ladder.

Model	LogLoss	AUC
1. Baseline (intercept)	0.6230	0.500
2. Treatment only	0.6215	0.530
3. + Vote history	0.5829	0.667
4. Full (+ age, sex, hh size)	0.5796	0.673

The ordering is instructive. The treatment dummies — the entire causal content of the experiment — move the AUC from 0.500 to only about 0.53: knowing which mailing someone received barely helps you guess *whether they will vote*, even though it shifts the *probability* causally. Vote history is where the predictive action is (AUC jumps to about 0.67), and demographics add a further increment. Big average effects and big predictive power are simply different things.

5.3 10-fold cross-validation of the ladder

A single split is one random draw; cross-validation uses every household in both roles. We assign each **household** (again: the clustering unit, not the individual) to one of $K = 10$ folds; each fold takes a turn as the test set. We pool the out-of-fold predictions so every individual is predicted exactly once, by a model that never saw their household.

```
K <- 10
set.seed(2008)
fold_of_hh <- sample(rep(1:K, length.out = length(hh_ids)))
ggl$fold <- fold_of_hh[match(ggl$hh_id, hh_ids)]
knitr::kable(t(as.matrix(table(Fold = ggl$fold))), caption =
  "Individuals per household-level fold (folds are equal in households, so individual
  ↪  counts vary slightly).")
```

Table 10: Individuals per household-level fold (folds are equal in households, so individual counts vary slightly).

1	2	3	4	5	6	7	8	9	10
34304	34418	34378	34366	34531	34345	34356	34570	34463	34353

```
n <- nrow(ggl)
cv_p <- matrix(NA_real_, n, 4,
               dimnames = list(NULL, c("base", "treat", "hist", "full")))
for (k in 1:K) {
  tri <- ggl$fold != k; tei <- !tri
  cv_p[tei, "base"]  <- mean(ggl$voted[tri])
  cv_p[tei, "treat"] <- predict(glm(f_treat, ggl[tri, ], family = binomial),
                                ggl[tei, ], type = "response")
  cv_p[tei, "hist"]  <- predict(glm(f_hist,  ggl[tri, ], family = binomial),
                                ggl[tei, ], type = "response")
  cv_p[tei, "full"]  <- predict(glm(f_full,  ggl[tri, ], family = binomial),
                                ggl[tei, ], type = "response")
}
cv_tab <- data.frame(
  Model   = split_tab$Model,
  CV_LogLoss = round(apply(cv_p, 2, logloss, y = ggl$voted), 4),
  CV_AUC     = round(apply(cv_p, 2, auc_of,  y = ggl$voted), 3),
  row.names = NULL)
knitr::kable(cv_tab, caption =
  "10-fold cross-validated (household folds, pooled out-of-fold) log-loss and AUC for the
  ↪  model ladder.")
```

Table 11: 10-fold cross-validated (household folds, pooled out-of-fold) log-loss and AUC for the model ladder.

Model	CV_LogLoss	CV_AUC
1. Baseline (intercept)	0.6237	0.497
2. Treatment only	0.6221	0.529
3. + Vote history	0.5848	0.664
4. Full (+ age, sex, hh size)	0.5813	0.671

The CV numbers agree with the single split to the third decimal — with 344,084 observations, evaluation noise is tiny — and the story is identical: each rung of the ladder helps, history helps most, and the full model wins on both metrics.

5.4 Bias–variance: a validation curve in the age polynomial

The turnout–age profile in the EDA was strongly curved, so *some* flexibility in age must help — but how much? We take the full model and sweep the **degree of the age polynomial** from 1 (a straight line in the log-odds; most rigid, most *bias*) upward until the cross-validated loss stops improving and turns back up (most *variance*), recording the **training** log-loss (fit and scored on all rows) and the **cross-validated** log-loss (household folds as above) at each degree.

```
degs <- 1:14
form_deg <- function(p)
  update(f_hist, as.formula(paste(
    ". ~ . + poly(age,", p, ") + female + hh_size")))
vc <- data.frame(degree = degs, Train = NA_real_, CV = NA_real_,
                 CV_se = NA_real_)
for (i in seq_along(degs)) {
  p <- degs[i]
  fit_all <- glm(form_deg(p), ggl, family = binomial)
  vc$Train[i] <- logloss(ggl$voted, fitted(fit_all))
  fold_ll <- numeric(K)
  for (k in 1:K) {
    tri <- ggl$fold != k; tei <- !tri
    fit <- glm(form_deg(p), ggl[tri, ], family = binomial)
    fold_ll[k] <- logloss(ggl$voted[tei],
                          predict(fit, ggl[tei, ], type = "response"))
  }
  vc$CV[i]    <- mean(fold_ll)
  vc$CV_se[i] <- sd(fold_ll) / sqrt(K)
}
best_deg <- vc$degree[which.min(vc$CV)]
# one-standard-error rule: simplest degree whose CV loss is within one SE
# of the minimum -- the standard guard against chasing noise in a flat region
thresh   <- min(vc$CV) + vc$CV_se[which.min(vc$CV)]
best_1se <- min(vc$degree[vc$CV <= thresh])
vl <- rbind(data.frame(degree = vc$degree, LogLoss = vc$Train,
                       Curve = "Training (in-sample)"),
            data.frame(degree = vc$degree, LogLoss = vc$CV,
                       Curve = "Cross-validated (held-out)"))
ggplot(vl, aes(degree, LogLoss, colour = Curve)) +
  geom_line(linewidth = 0.8) + geom_point() +
  geom_errorbar(data = data.frame(degree = vc$degree, LogLoss = vc$CV,
                    Curve = "Cross-validated (held-out)",
```

```
                  lo = vc$CV - vc$CV_se, hi = vc$CV + vc$CV_se),
              aes(ymin = lo, ymax = hi), width = 0.25, linewidth = 0.4) +
  geom_vline(xintercept = best_deg,  linetype = "dashed", colour = "grey40") +
  geom_vline(xintercept = best_1se, linetype = "dotted", colour = "grey20") +
  scale_x_continuous(breaks = degs) +
  labs(x = "age polynomial degree --- more flexible to the right",
       y = "log-loss", colour = NULL,
       title = paste0("CV minimum at degree ", best_deg,
                      " (dashed); one-SE choice degree ", best_1se,
                      " (dotted)")) +
  theme_minimal(base_size = 11) + theme(legend.position = "top")
```

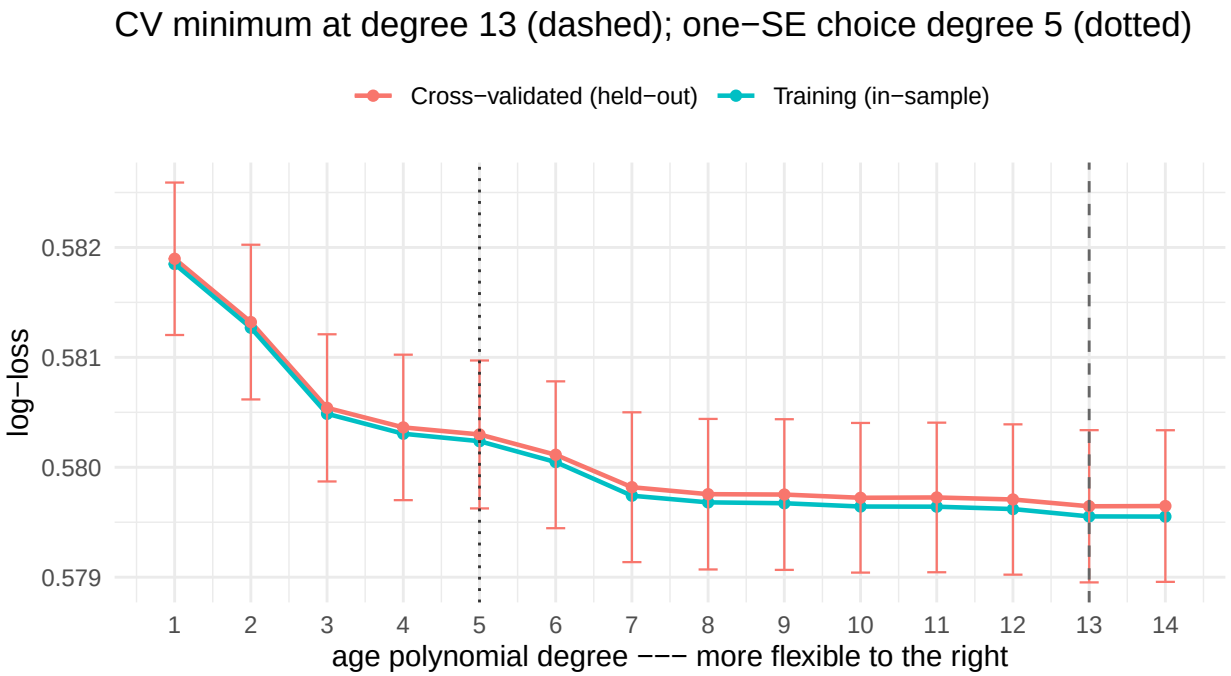


Figure 7: Validation curve for the age-polynomial degree in the full logit. Training log-loss falls monotonically with flexibility; CV log-loss drops sharply through degree 2-3, reaches its minimum (dashed line) in a flat region, and then turns back up – the right-hand side of the U is real even at n = 344,084, just shallow. The sweep continues past the minimum until the upturn is visible.

```
knitr::kable(data.frame(degree = vc$degree,
                        Train_LogLoss = round(vc$Train, 5),
                        CV_LogLoss    = round(vc$CV, 5)),
             caption = "Validation-curve values: training vs. cross-validated log-loss by
             ↪  age-polynomial degree.")
```

Table 12: Validation-curve values: training vs. cross-validated log-loss by age-polynomial degree.

degree	Train_LogLoss	CV_LogLoss
1	0.58185	0.58190
2	0.58127	0.58132
3	0.58049	0.58054

degree	Train_LogLoss	CV_LogLoss
4	0.58031	0.58036
5	0.58024	0.58030
6	0.58005	0.58011
7	0.57974	0.57982
8	0.57968	0.57976
9	0.57967	0.57975
10	0.57964	0.57972
11	0.57964	0.57973
12	0.57962	0.57971
13	0.57955	0.57965
14	0.57955	0.57965

Three lessons. First, the familiar mechanics: training loss falls monotonically with degree (flexibility always fits the data it sees at least as well), while CV loss drops sharply as the polynomial captures the real curvature in the age profile, bottoms out at degree 13, and **turns back up** as still-higher degrees chase noise — we deliberately extend the sweep past the minimum until that upturn is visible. Second, the minimum sits in a *flat, noisy region*: several adjacent degrees are within one standard error of it (the error bars). The standard response is the **one-standard-error rule** — choose the *simplest* model whose CV loss is within one SE of the minimum — which selects degree 5. That is the model we carry forward. Third, a lesson about **sample size and variance**: at n = 344,084 the upturn is shallow, because overfitting is a *ratio* of flexibility to data. Shrink the data and the same sweep produces the textbook U. The next figure repeats the exercise on a random subsample of roughly 1,500 households ($n \approx 2{,}900$ individuals):

```
set.seed(7)
hh_sub  <- sample(unique(ggl$hh_id), 1500)
sub     <- ggl[ggl$hh_id %in% hh_sub, ]
fold_s  <- sample(rep(1:K, length.out = length(hh_sub)))
sub$fold <- fold_s[match(sub$hh_id, hh_sub)]
degs_s  <- 1:12
vcs <- data.frame(degree = degs_s, Train = NA_real_, CV = NA_real_)
for (i in seq_along(degs_s)) {
  p <- degs_s[i]
  fit_all <- suppressWarnings(glm(form_deg(p), sub, family = binomial))
  vcs$Train[i] <- logloss(sub$voted, fitted(fit_all))
  oof <- numeric(nrow(sub))
  for (k in 1:K) {
    tri <- sub$fold != k
    fit <- suppressWarnings(glm(form_deg(p), sub[tri, ], family = binomial))
    oof[!tri] <- suppressWarnings(
      predict(fit, sub[!tri, ], type = "response"))
  }
  vcs$CV[i] <- logloss(sub$voted, oof)
}
vls <- rbind(data.frame(degree = vcs$degree, LogLoss = vcs$Train,
                        Curve = "Training (in-sample)"),
             data.frame(degree = vcs$degree, LogLoss = vcs$CV,
                        Curve = "Cross-validated (held-out)"))
ggplot(vls, aes(degree, LogLoss, colour = Curve)) +
  geom_line(linewidth = 0.8) + geom_point() +
  geom_vline(xintercept = vcs$degree[which.min(vcs$CV)],
             linetype = "dashed", colour = "grey40") +
  scale_x_continuous(breaks = degs_s) +
```

```
  labs(x = "age polynomial degree", y = "log-loss", colour = NULL,
       title = sprintf(
         "Same sweep, n = %s: the classic U (CV minimum at degree %d)",
         format(nrow(sub), big.mark = ","),
         vcs$degree[which.min(vcs$CV)])) +
  theme_minimal(base_size = 11) + theme(legend.position = "top")
```

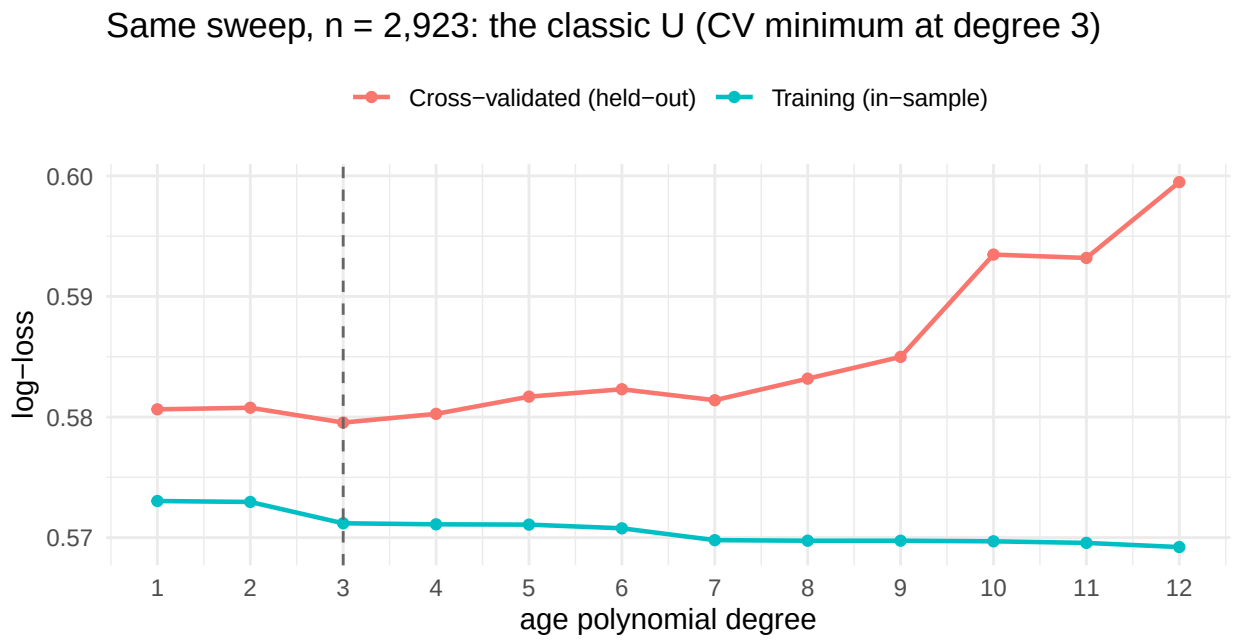


Figure 8: The same validation curve on a random subsample of about 2,900 individuals. With 1 percent of the data, the classic U appears in force: CV log-loss is minimized at a much lower degree and blows up quickly beyond it, while training loss keeps falling. Overfitting is a ratio of flexibility to data.

The chosen degree is not an ornament: **we carry the one-SE model forward** and everything in the next two sections — ROC, precision–recall, and calibration — is that model, refit on the training households only and scored on the untouched test households.

5.5 Testing the CV-chosen model out of sample: ROC and precision–recall

Does the degree chosen by cross-validation actually validate on data it never touched? Before the curves, the direct check — test-set log-loss and AUC for the rigid (degree-1), one-SE-chosen (degree-5), CV-minimum (degree-13), and most-flexible (degree-14) versions of the full model, each fit on training households only:

```
deg_check <- unique(c(1, best_1se, best_deg, 14))
tab <- data.frame(degree = deg_check, Test_LogLoss = NA_real_,
                  Test_AUC = NA_real_)
for (i in seq_along(deg_check)) {
  m <- suppressWarnings(
    glm(form_deg(deg_check[i]), train, family = binomial))
  pp <- suppressWarnings(predict(m, test, type = "response"))
  tab$Test_LogLoss[i] <- logloss(test$voted, pp)
  tab$Test_AUC[i] <- as.numeric(
    pROC::auc(pROC::roc(test$voted, pp, quiet = TRUE, direction = "<")))
}
knitr::kable(tab, digits = 5,
```

```
  caption = "Out-of-sample check of the CV choice on untouched test households: the
  ↪  one-SE degree matches the CV-minimum degree to the fourth decimal at a fraction of
  ↪  the flexibility; degree 14 buys nothing.")
```

Table 13: Out-of-sample check of the CV choice on untouched test households: the one-SE degree matches the CV-minimum degree to the fourth decimal at a fraction of the flexibility; degree 14 buys nothing.

degree	Test_LogLoss	Test_AUC
1	0.58025	0.67201
5	0.57861	0.67548
13	0.57786	0.67642
14	0.57787	0.67639

The one-SE model performs indistinguishably from the CV minimum on the untouched test households at a fraction of the flexibility — cross-validation plus the one-SE rule did their job. We now evaluate this model (full logit, age degree 5) on the held-out household test set from the single split, starting with the ROC curve: the tradeoff between the true-positive rate and the false-positive rate as the classification threshold sweeps from 1 to 0.

```
m_best  <- glm(form_deg(best_1se), train, family = binomial)
p_best  <- predict(m_best, test, type = "response")
roc_best <- pROC::roc(test$voted, p_best,  quiet = TRUE, direction = "<")
roc_tr   <- pROC::roc(test$voted, p_tr,    quiet = TRUE, direction = "<")
plot(roc_best, col = "steelblue", lwd = 2, legacy.axes = TRUE,
     main = sprintf("ROC, held-out test set (full AUC = %.3f)",
                    as.numeric(pROC::auc(roc_best))))
plot(roc_tr, col = "firebrick", lwd = 2, add = TRUE)
legend("bottomright", lwd = 2, col = c("steelblue", "firebrick"),
       legend = c(sprintf("Full model (AUC = %.3f)",
                          as.numeric(pROC::auc(roc_best))),
                  sprintf("Treatment only (AUC = %.3f)",
                          as.numeric(pROC::auc(roc_tr)))), bty = "n")
```

With a base rate of 31.5% — roughly one voter for every two non-voters — the ROC curve can flatter a model, because the false-positive *rate* has a large denominator of non-voters. The **precision–recall curve** asks the question a campaign would ask: *of the people the model flags as likely voters, what share actually vote (precision), and what share of all actual voters do we capture (recall)?*

```
ord  <- order(p_best, decreasing = TRUE)
yy   <- test$voted[ord]
tp   <- cumsum(yy)
prec <- tp / seq_along(yy)
rec  <- tp / sum(yy)
thin <- seq(1, length(yy), by = 200)      # thin for plotting only
pr <- data.frame(recall = rec[thin], precision = prec[thin])
base_rate <- mean(test$voted)
ggplot(pr, aes(recall, precision)) +
  geom_line(colour = "steelblue", linewidth = 0.9) +
  geom_hline(yintercept = base_rate, linetype = "dashed",
             colour = "grey40") +
  annotate("text", x = 0.15, y = base_rate - 0.035, size = 3.2,
```

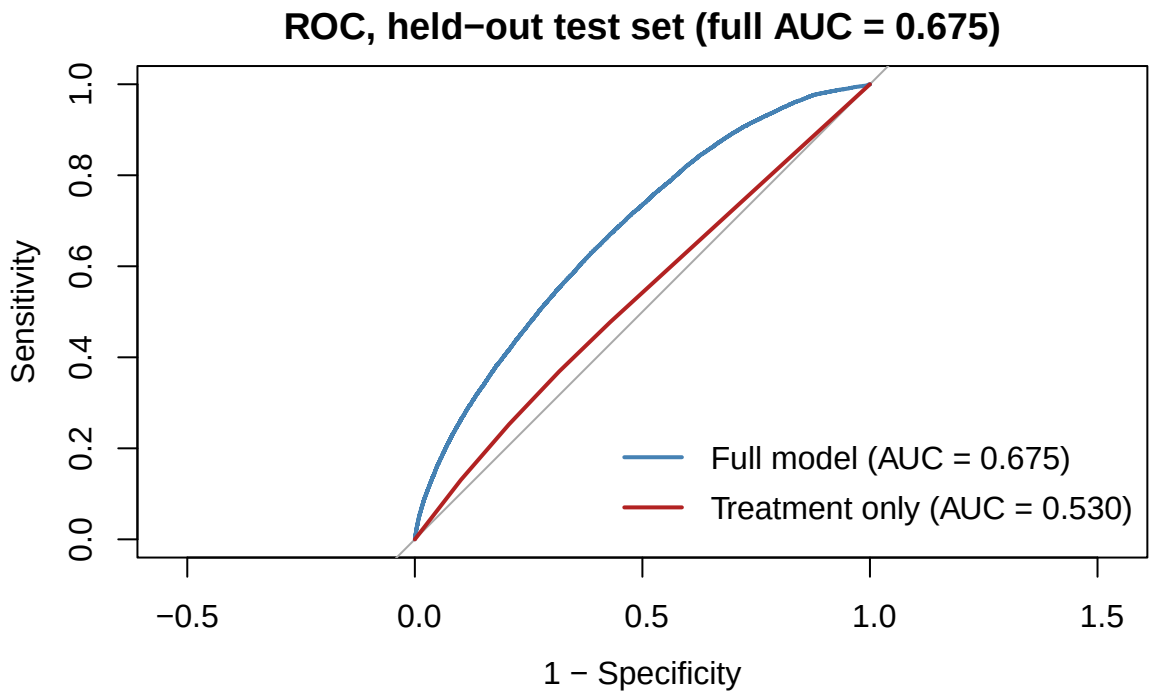


Figure 9: ROC curves on the held-out test set. The full model dominates the treatment-only model everywhere; the treatment-only curve hugs the diagonal, showing that experimental assignment alone barely ranks voters above non-voters.

```
          label = sprintf("base rate = %.3f", base_rate)) +
  ylim(0, 1) +
  labs(x = "Recall (share of actual voters captured)",
       y = "Precision (share of flagged who vote)",
       title = "Precision-recall, full model, held-out test set") +
  theme_minimal(base_size = 11)
```

Among the model's top-ranked prospects, precision is high — the top 5% of the test set votes at 65% — more than double the base rate. Any threshold is a business decision: a campaign with money for one mailing per ten households wants the high-precision end; a canvass that must reach every plausible voter accepts lower precision for high recall.

5.6 Calibration and the Brier score

AUC only checks *ranking*. **Calibration** checks the probabilities themselves: among people to whom the model assigns, say, a 40% chance of voting, do about 40% actually vote? We bin the test set into deciles of predicted probability and plot the mean prediction against the observed turnout in each bin. Experimental data makes this exercise especially clean: the predictions embed causal treatment contrasts we *know* are right on average, so a well-specified model should sit on the 45-degree line.

```
dec <- cut(p_best, quantile(p_best, 0:10 / 10), include.lowest = TRUE)
cal <- data.frame(
  mean_pred = tapply(p_best, dec, mean),
  obs_rate  = tapply(test$voted, dec, mean),
  n         = as.integer(table(dec)))
ggplot(cal, aes(mean_pred, obs_rate)) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed",
```

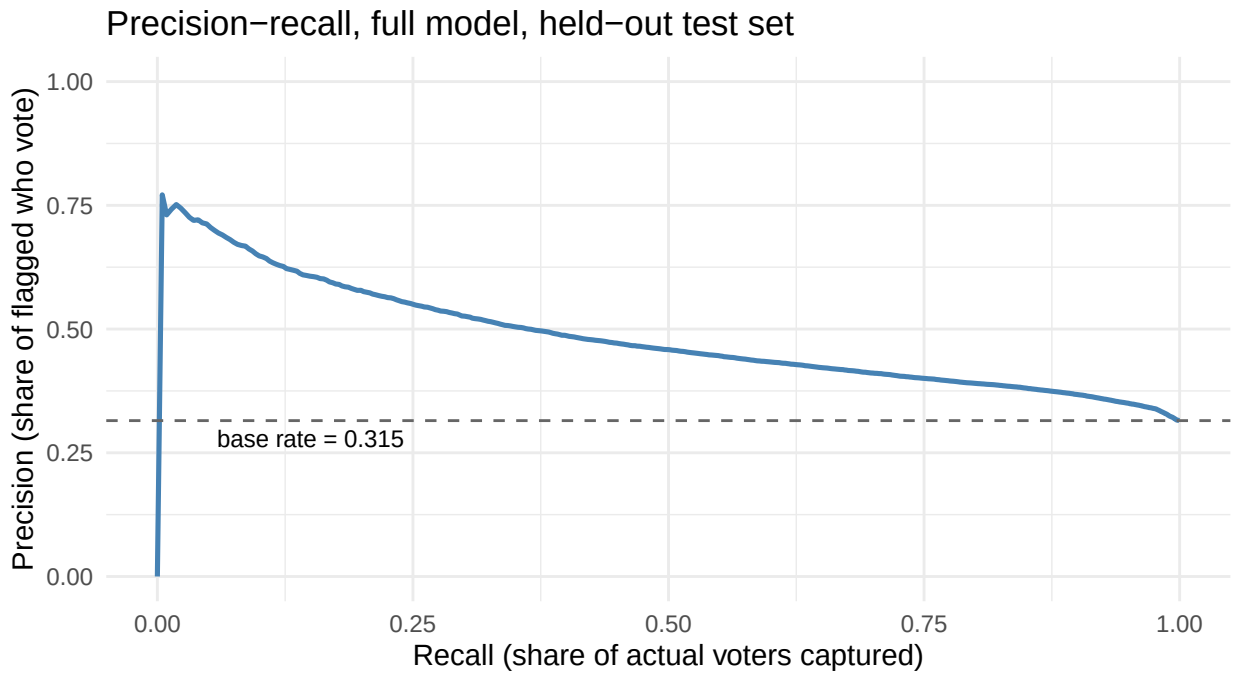


Figure 10: Precision-recall on the held-out test set. The dashed line is the base rate (~32 percent), the precision of flagging everyone. The full model reaches roughly 80 percent precision on its most confident picks and degrades gracefully toward the base rate as recall approaches 1.

```
            colour = "grey50") +
  geom_point(colour = "steelblue", size = 2.4) +
  geom_line(colour = "steelblue", linewidth = 0.6) +
  coord_equal(xlim = c(0, 0.8), ylim = c(0, 0.8)) +
  labs(x = "Mean predicted P(vote), decile of prediction",
       y = "Observed turnout in decile",
       title = "Calibration: predicted vs. observed by decile") +
  theme_minimal(base_size = 11)
```

```
knitr::kable(data.frame(Decile = 1:10,
                        Mean_pred = round(cal$mean_pred, 3),
                        Observed  = round(cal$obs_rate, 3),
                        N = format(cal$n, big.mark = ",")),
             row.names = FALSE, caption =
  "Decile reliability table for the full model on the held-out test set.")
```

Table 14: Decile reliability table for the full model on the held-out test set.

Decile	Mean_pred	Observed	N
1	0.108	0.086	10,341
2	0.170	0.172	10,323
3	0.215	0.223	10,322
4	0.258	0.276	10,335
5	0.292	0.299	10,327
6	0.327	0.324	10,330
7	0.364	0.355	10,331

Decile	Mean_pred	Observed	N
8	0.405	0.386	10,322
9	0.462	0.444	10,326
10	0.572	0.585	10,329

```
brier <- data.frame(
  Model = c("Baseline (base rate)", "Treatment only",
            paste0("Full (age degree ", best_deg, ")")),
  Brier = round(c(mean((p_base - test$voted)^2),
                  mean((p_tr   - test$voted)^2),
                  mean((p_best - test$voted)^2)), 4))
knitr::kable(brier, caption =
  "Brier scores (mean squared error of the predicted probability) on the held-out test
  ↪  set; lower is better.")
```

Table 15: Brier scores (mean squared error of the predicted probability) on the held-out test set; lower is better.

Model	Brier
Baseline (base rate)	0.2158
Treatment only	0.2151
Full (age degree 13)	0.1974

The reliability curve tracks the diagonal from the ~10%-probability decile up to the ~70% decile: when this model says 60%, about 60% vote. The Brier score summarizes the same thing in one number, and its ladder mirrors the log-loss ladder — most of the improvement over the base-rate benchmark comes from vote history and age, not from treatment.

5.7 What ML adds — and what it does not

The closing contrast of Day 1. The **experiment** delivers something no predictive model can: an unbiased estimate of what *would happen* if we mailed the Neighbors letter to anyone — +8.1 points, full stop, no modeling assumptions. The **predictive models** deliver something the experiment alone does not: individual-level probabilities — *this* voter is a 9% prospect, *that* one is 71% — driven overwhelmingly by vote history and age, variables the experiment never manipulated and which no one can manipulate. The two outputs answer different questions and are useful together: a modern GOTV campaign uses the *causal* estimate to decide whether the mailing is worth sending and the *predictive* model to decide whom to send it to (don't waste postage on sure-thing voters or hopeless abstainers). Keeping "predicts well" and "identifies a causal effect" distinct — and knowing which one your model is delivering — is the most important habit this course teaches.

6 Independent exercises (each about 10 minutes)

1. **Leaky splits.** Make a 70/30 split that samples *individuals* (rows of `ggl`) instead of households, refit the full model (`glm(f_full, ...)`), and recompute test log-loss and AUC with `logloss()` and `auc_of()`. Which direction do the scores move relative to the household-split table, and why is the household split the honest one?
2. **Grid search vs. random search.** The validation curve above is an *exhaustive grid search* over all 14 age-polynomial degrees; the slides contrast this with *random search* under a smaller budget. Using the already-computed `vc` table, simulate a random search that evaluates only 4 degrees:

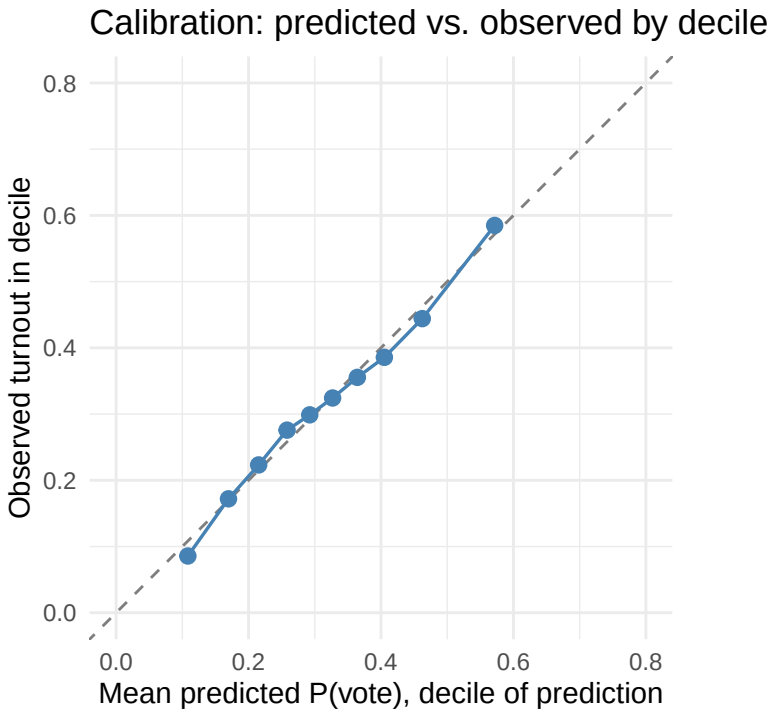


Figure 11: Reliability curve on the held-out test set: mean predicted probability vs. observed turnout within deciles of the prediction. Points sit on the 45-degree line across the whole range – the logit's probabilities mean what they say.

`min(vc$CV[vc$degree %in% sample(vc$degree, 4)])`. Repeat it 20 times with `replicate()`. How often does the 4-draw search land within one standard error of the exhaustive minimum?

3. **Accuracy vs. the base rate.** Using the full model's test-set predictions `p_fu`, compute classification accuracy at a 0.5 threshold, and compare it with the accuracy of the no-model rule "predict that nobody votes." What does the comparison say about accuracy as a metric when the base rate is near 30%?